

WEEK 2: Product and Category Management System

- ATCHAYA S [E25A1106]

Scenario:

The customized phone case business wants to maintain information about **phone cases and their customization details**. Each phone case belongs to a specific category or phone model, and the system should manage product details such as **name, price, and stock availability**.

1. Category and Product Tables

Category table:

```
SQL> CREATE TABLE Category(Category_ID VARCHAR2(7),
 2   Category_Name VARCHAR2(20),
 3   Description VARCHAR2(50));
Table created.
```

Product table:

```
SQL> CREATE TABLE Product(Product_ID VARCHAR2(7),
 2   Product_Name VARCHAR2(20),
 3   Category_ID VARCHAR2(7),
 4   Price NUMBER(9,2),
 5   Stock_Quantity Number);
Table created.
```

```
SQL> desc Category;
Name                                Null?      Type
-----
CATEGORY_ID                        VARCHAR2(7)
CATEGORY_NAME                      VARCHAR2(20)
DESCRIPTION                        VARCHAR2(50)

SQL> desc Product;
Name                                Null?      Type
-----
PRODUCT_ID                        VARCHAR2(7)
PRODUCT_NAME                      VARCHAR2(20)
CATEGORY_ID                       VARCHAR2(7)
PRICE                            NUMBER(9,2)
STOCK_QUANTITY                    NUMBER
```

Category and product Tables created.

Now, we have to find the relationship between the tables and apply constraints to the tables.

2. Define Relationships and Constraints

Relationship:

One Category can have many Products.

The relationship between Category and Product is One-to-Many (1 → *).

We identified that Category and Product have a relationship. Now, we need to find which key is used to connect these two tables.

Key Constraints:

PRIMARY KEY

A **Primary Key** is a column that uniquely identifies each row in a table.

In the **Category table**, Category_ID is unique, so it is the Primary Key.

In the **Product table**, Product_ID is unique, so it is the Primary Key.

FOREIGN KEY

A **Foreign Key** is a key that connects two tables.

Here, **Category_ID** from the Category table is used as a Foreign Key in the Product table.

NOT NULL

NOT NULL means a column cannot have an empty (NULL) value.

For example, the Product Name or Category Name should not be empty.

CHECK

CHECK is used to make sure that the value follows a given condition.

For example, the Price of a product should always be greater than 0.

UNIQUE

UNIQUE ensures that duplicate values are not allowed in a column.

For example, if **Category_Name** has a UNIQUE constraint, two categories cannot have the same name.

Defining Relationship and Creating Constraints for Product and Category:

Primary key:

```
SQL> ALTER TABLE Category ADD CONSTRAINT PK_Category PRIMARY KEY (category_ID);
Table altered.

SQL> ALTER TABLE Product ADD CONSTRAINT PK_Product PRIMARY KEY (product_ID);
Table altered.
```

Creating relationship with using FOREIGN KEY:

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT FK_Product_Category
  3  FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);
```

Adding Constraints:

NOT NULL:

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;
Table altered.

SQL> ALTER TABLE Product  MODIFY Product_Name VARCHAR(100) NOT NULL;
Table altered.
```

CHECK:

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Price CHECK (Price > 0);
Table altered.

SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);
Table altered.
```

UNIQUE:

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);
Table altered.
```

3. INSERTING VALUES INTO TABLES

Category Table:

```
SQL> INSERT INTO Category
  2  VALUES ('CAT0001','Basic Cases','Simple and affordable phone cases');

1 row created.

SQL> INSERT INTO Category
  2  VALUES ('CAT0002','Silicone Cases','Soft and flexible silicone cases');

1 row created.

SQL> INSERT INTO Category
  2  VALUES ('CAT0003','Transparent Cases','Clear transparent phone cases');

1 row created.

SQL> INSERT INTO Category
  2  VALUES ('CAT0004','Printed Cases','Customized printed phone cases');

1 row created.

SQL> INSERT INTO Category
  2  VALUES ('CAT0005','Designer Cases','Premium designer phone cases');

1 row created.

SQL> INSERT INTO Category
  2  VALUES ('CAT0006','Photo Cases','Personalized photo phone cases');

1 row created.

SQL> INSERT INTO Category
  2  VALUES ('CAT0007','Protective Cases','Shockproof and durable phone cases');

1 row created.
```

Product Table:

```
SQL> INSERT INTO Product
  2  VALUES ('P000001','Clear Shock Case','CAT0001',299.00,50);

1 row created.

SQL> INSERT INTO Product
  2  VALUES ('P000002','Soft Silicone Case','CAT0002',249.00,45);

1 row created.

SQL> INSERT INTO Product
  2  VALUES ('P000003','Crystal Clear Case','CAT0003',199.00,60);

1 row created.

SQL> INSERT INTO Product
  2  VALUES ('P000004','Floral Printed Case','CAT0004',349.00,35);

1 row created.

SQL> INSERT INTO Product
  2  VALUES ('P000005','Luxury Designer Case','CAT0005',599.00,25);

1 row created.

SQL> INSERT INTO Product
  2  VALUES ('P000006','Personal Photo Case','CAT0006',449.00,40);

1 row created.

SQL> INSERT INTO Product
  2  VALUES ('P000007','Heavy Duty Case','CAT0007',499.00,30);

1 row created.
```

After Insertion Category Table:

CATEGORY_NAME
DESCRIPTION
Customized printed phone cases
CAT0005
Designer Cases
Premium designer phone cases
CAT0006
CATEGOR
CATEGORY_NAME
DESCRIPTION
Photo Cases
Personalized photo phone cases
CAT0007
Protective Cases
Shockproof and durable phone cases
7 rows selected.

After Insertion Product Table:

PRODUCT_NAME		
CATEGOR	PRICE	STOCK_QUANTITY
CAT0004	349	35
P000005		
Luxury Designer Case		
CAT0005	599	25
P000006		
PRODUCT		

PRODUCT_NAME		
CATEGOR	PRICE	STOCK_QUANTITY
Personal Photo Case		
CAT0006	449	40
P000007		
Heavy Duty Case		
CAT0007	499	30
7 rows selected.		

CRUD Operations:

Insertion record

```
SQL> INSERT INTO Product (Product_ID, Product_Name, Category_ID, Price, Stock_Quantity)
  2  VALUES ('P000008', 'Marble Phone Case', 'CAT0004', 399, 20);

1 row created.
```

Update Price

```
SQL> UPDATE Product SET Price = 449 WHERE Product_Name = 'Marble Phone Case';

1 row updated.
```

Deletion

```
SQL> DELETE FROM Product WHERE Product_Name = 'Marble Phone Case';

1 row deleted.
```

4. Category-Wise Product Reports

SQL query:

```
SELECT c.CATEGORY_ID,
       c.CATEGORY_NAME,
       p.PRODUCT_ID,
       p.PRODUCT_NAME,
       p.PRICE,
       p.STOCK_QUANTITY
FROM Category c
JOIN Product p
ON c.CATEGORY_ID = p.CATEGORY_ID
ORDER BY c.CATEGORY_ID;
```

```

SQL> SELECT c.CATEGORY_ID,
2         c.CATEGORY_NAME,
3         p.PRODUCT_ID,
4         p.PRODUCT_NAME,
5         p.PRICE,
6         p.STOCK_QUANTITY
7 FROM Category c
8 JOIN Product p
9 ON c.CATEGORY_ID = p.CATEGORY_ID
10 ORDER BY c.CATEGORY_ID;

```

```

CATEGORY
-----
CATEGORY_NAME
-----
PRODUCT
-----
PRODUCT_NAME
-----
PRICE STOCK_QUANTITY
-----
CAT00002
Clothing
P10002

CATEGORY
-----
CATEGORY_NAME
-----
PRODUCT
-----
PRODUCT_NAME
-----
PRICE STOCK_QUANTITY
-----
Croptops and Shirt
2000          35

```

```

CATEGORY_NAME
-----
PRODUCT
-----
PRODUCT_NAME
-----
PRICE STOCK_QUANTITY
-----
Personal Photo Case
449          40

CATEGOR
-----
CATEGORY_NAME
-----
PRODUCT
-----
PRODUCT_NAME
-----
PRICE STOCK_QUANTITY
-----
CAT0007
Protective Cases
P000007

CATEGOR
-----
CATEGORY_NAME
-----
PRODUCT
-----
PRODUCT_NAME
-----
PRICE STOCK_QUANTITY
-----
Heavy Duty Case
499          30

7 rows selected.

```

Counting Number of products in each Category:

```
SQL> SELECT Category_ID, COUNT(*) AS Number_of_Products
  2  FROM Product
  3  GROUP BY Category_ID;
```

CATEGOR	NUMBER_OF_PRODUCTS
CAT0003	4
CAT0001	6
CAT0002	4
CAT0006	4
CAT0007	4
CAT0005	4
CAT0004	4

7 rows selected.

Finding highest priced product in each category:

```
SQL> SELECT Category_ID, MAX(Price) AS Highest_Price
  2  FROM Product
  3  GROUP BY Category_ID;
```

CATEGOR	HIGHEST_PRICE
CAT0003	269
CAT0001	299
CAT0002	299
CAT0006	499
CAT0007	599
CAT0005	699
CAT0004	379

7 rows selected.

Display categories having more than 5 products

```
SQL> SELECT Category_ID, COUNT(*) AS Number_of_Products
  2  FROM Product
  3  GROUP BY Category_ID
  4  HAVING COUNT(*) > 5;
```

CATEGOR	NUMBER_OF_PRODUCTS
CAT0001	6

Find average product price category-wise

```
SQL> SELECT Category_ID, AVG(Price) AS Average_Price  
2 FROM Product  
3 GROUP BY Category_ID;
```

CATEGOR	AVERAGE_PRICE
CAT0003	236.5
CAT0001	239
CAT0002	276.5
CAT0006	469
CAT0007	536.5
CAT0005	624
CAT0004	351.5

7 rows selected.